VOL. 49

AI Agents

Building autonomous AI agents with tools and reasoning

Source	huggingface.co/learn/agents-course
Generated	2026-10-11
Level	Intermediate
Estimated study time	20 hours
Language	English
Volume	Vol. 49 - AI Agents

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume . . . 3

Chapter 1 - What is an AI agent? . . . 3

1.1 Summary . . . 3

1.2 Concepts . . . 3

1.3 Examples . . . 3

Example 1 - Simple reflex agent . . . 3

1.4 Mini test . . . 4

Chapter 2 - Tools and tool use . . . 4

2.1 Summary . . . 4

2.2 Concepts . . . 5

2.3 Examples . . . 5

Example 1 - Tool schema . . . 5

Example 2 - Tool use loop . . . 5

2.4 Mini test . . . 6

Chapter 3 - Memory and context . . . 6

3.1 Summary . . . 6

3.2 Mini test . . . 6

Chapter 4 - Planning and reasoning . . . 7

4.1 Summary . . . 7

4.2 Mini test . . . 7

Final test . . . 7

5.1 Final test answers . . . 7

Sources . . . 8

How to use this volume

Prerequisites. Familiarity with LLMs, basic Python programming.

Estimated study time. About 20 hours.

Chapter 1 - What is an AI agent?

1.1 Summary

- An agent is a system that can perceive its environment and act autonomously.
- Components: goal, sensors, actuators, policy (decision-making).
- Key capability: using tools to interact with the world beyond text.
- Types: reflex (no state), goal-based, utility-based, learning agents.
- Modern LLM agents: receive goals, use tools, maintain memory.

1.2 Concepts

Component	Description
Goal	What the agent aims to achieve
Tools	Functions the agent can call
Memory	Persistent state across interactions
Planning	Breaking goals into subgoals
Perception	Reading and interpreting environment state

1.3 Examples

Example 1 - Simple reflex agent

Python

```python
# ch01_reflex_agent.py
class SimpleVacuumAgent:
    def __init__(self):
        self.location = "A"
        self.environment = {"A": "dirty", "B": "clean"}

    def perceive(self):
        return self.environment[self.location]

    def act(self):
        if self.perceive() == "dirty":
            action = "clean"
            self.environment[self.location] = "clean"
        else:
            if self.location == "A":
                self.location = "B"
            else:
                self.location = "A"
            action = "move"
        return f"Location {self.location}: {action}"

agent = SimpleVacuumAgent()
for i in range(5):
    print(f"Step {i+1}: {agent.act()}")
```

output

```
Step 1: Location A: clean
Step 2: Location B: move
Step 3: Location B: move
Step 4: Location A: move
Step 5: Location A: move
```

1.4 Mini test

1. What are the four components of an agent? 2. What is a tool in an LLM agent? 3. What differentiates an agent from a chatbot?

Answers: 1) Goal, sensors/actuators, policy, memory. 2) A callable function. 3) Agents act on the world, chatbots only respond.

Chapter 2 - Tools and tool use

2.1 Summary

- Tools extend an agent's capabilities beyond text generation.
- Common tools: search, calculator, database, API, file system, code interpreter.
- Tool calling: the model outputs a tool name + arguments in a structured format.
- The agent executes the tool and feeds the result back.
- Tool descriptions (schemas) must be clear and unambiguous.

2.2 Concepts

Tool	Purpose
Search	Query the web for current information
Calculator	Perform math beyond model accuracy
Code execution	Run code, validate logic
API calls	Interact with external services
File read/write	Access documents

2.3 Examples

Example 1 - Tool schema

JSON

```json
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get current weather for a location",
    "parameters": {
      "type": "object",
      "properties": {
        "location": {
          "type": "string",
          "description": "City name, e.g., San Francisco"
        },
        "unit": {
          "type": "string",
          "enum": ["celsius", "fahrenheit"]
        }
      },
      "required": ["location"]
    }
  }
}
```

Example 2 - Tool use loop

Python

```python
# ch02_tool_loop.py
# Simplified agentic loop structure

def agent_loop(task, tools, llm):
    """
    An agentic loop repeats:
    1. Receive task + context
    2. Decide: respond, or call tools
    3. If tool calls: execute tools, append results
    4. Repeat until response or max turns
    """

    messages = [{"role": "user", "content": task}]
    for _ in range(10):  # max turns
        response = llm(messages, tools)

        if response.tool_calls:
            for call in response.tool_calls:
                result = execute_tool(call, tools)
                messages.append({"role": "tool", "content": result})
            messages.append(response)
        else:
            return response.content

    return "Max turns reached"
```

2.4 Mini test

1. What is a tool schema? 2. Why are tools important for agents? 3. What does `required` mean in a schema?

Answers: 1) Defines tool parameters and types. 2) They let agents interact with the real world. 3) Mandatory parameters the caller must provide.

Chapter 3 - Memory and context

3.1 Summary

- Short-term memory: current conversation context.
- Long-term memory: persistent knowledge across sessions.
- Context window: maximum tokens the model can process.
- Strategies: summarisation, external storage, retrieval-augmented generation.

3.2 Mini test

1. What limits short-term memory? 2. How to implement long-term memory? 3. What is RAG?

Answers: 1) Context window size. 2) Vector database, summaries, files. 3) Retrieval-augmented generation: fetch relevant docs before answering.

Chapter 4 - Planning and reasoning

4.1 Summary

- Planning breaks complex goals into sequential steps.
- Techniques: chain-of-thought, tree-of-thoughts, ReAct.
- ReAct: combine reasoning (chain-of-thought) with acting (tool use).
- Agents can self-correct when tools reveal mistakes.

4.2 Mini test

1. What is ReAct? 2. How does chain-of-thought help? 3. What is tree-of-thoughts?

Answers: 1) Combine reasoning + acting in a loop. 2) Makes multi-step reasoning explicit. 3) Explore multiple reasoning paths in parallel.

Final test

1. What are the four components of an agent?
2. What is a tool in an LLM agent?
3. What differentiates an agent from a chatbot?
4. What is a tool schema?
5. What is the context window?
6. What does ReAct combine?
7. What is chain-of-thought?
8. What is retrieval-augmented generation (RAG)?
9. What is short-term memory?
10. What is planning in AI agents?

5.1 Final test answers

1. Goal, sensors/actuators, policy, memory. (Or: perception, decision, action, memory.)
2. A callable function that extends capabilities.
3. Agents can act on the environment, not just respond.
4. A specification of tool parameters and types.
5. The maximum number of tokens the model can process at once.
6. Reasoning and acting (tool use) in an interleaved loop.
7. Asking the model to reason step by step before answering.
8. Fetching relevant documents before generating a response.

9. The current conversation context within the context window.
10. Breaking a complex goal into a sequence of executable steps.

Sources

- https://huggingface.co/learn/agents-course
- https://huggingface.co/docs/transformers/main_classes/text_generation
- https://langchain.com/